



Question 1: Contains Duplicate

Given an integer array `nums`, return `true` if any value appears **more than once** in the array, otherwise return `false`.

Example 1:

Java

Input: `nums = [1, 2, 3, 3]`

Output: `true`

Example 2:

Java

Input: `nums = [1, 2, 3, 4]`

Output: `false`

Approach 1: Brute-force

1. Loop through the array with index `i` from `0` to `n - 1`
2. For each `i`, loop through the array with index `j` from `i + 1` to `n - 1`
3. If `nums[i] == nums[j]`, return `true`
4. If all comparisons finish and no duplicates are found, return `false`

```
1  class Solution {  
2      public boolean hasDuplicate(int[] nums) {  
3          int n = nums.length;  
4  
5          for (int i = 0; i < n; i++) {  
6              for (int j = i + 1; j < n; j++) {  
7                  if (nums[i] == nums[j]) {  
8                      return true;  
9                  }  
10             }  
11         }  
12  
13         return false;  
14     }  
15 }  
16 }
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Approach 2: Better

1. Sort the array
2. Traverse the array from index 0 to $n - 2$
3. If $nums[i] == nums[i + 1]$, return true
4. If no adjacent duplicates are found, return false

```
1  class Solution {  
2      public boolean hasDuplicate(int[] nums) {  
3          Arrays.sort(nums);  
4  
5          for (int i = 0; i < nums.length - 1; i++) {  
6              if (nums[i] == nums[i + 1]) {  
7                  return true;  
8              }  
9          }  
10  
11         return false;  
12     }  
13 }
```

Time Complexity: $O(n \log n)$ (due to sorting)

Space Complexity: $O(1)$ (ignoring sorting internals)

Approach 3: Optimal

1. Create an empty **HashSet**
2. Traverse the array
3. If the current number exists in the set, return **true**
4. Add the number to the set
5. If traversal ends, return **false**

```
1  class Solution {  
2      public boolean hasDuplicate(int[] nums) {  
3          Set<Integer> seen = new HashSet<>();  
4  
5          for (int num : nums) {  
6              if (seen.contains(num)) {  
7                  return true;  
8              }  
9              seen.add(num);  
10         }  
11  
12         return false;  
13     }  
14 }
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Question 2: Valid Anagram

Given two strings *s* and *t*, return true if the two strings are anagrams of each other, otherwise return false.

An anagram is a string that contains the exact same characters as another string, but the order of the characters can be different.

Example 1:

Input: *s* = "racecar", *t* = "carrace"

Output: true

Example 2:

Input: *s* = "jar", *t* = "jam"

Output: false

Constraints:

s and *t* consist of lowercase English letters.

Approach 1: Brute-force

Sort both strings and compare them.

If they are anagrams, their sorted versions will be identical.

```
2  ✓ public boolean isAnagram(String s, String t) {  
3  ✓     if (s.length() != t.length()) {  
4         return false;  
5     }  
6  
7     char[] sArr = s.toCharArray();  
8     char[] tArr = t.toCharArray();  
9  
10    Arrays.sort(sArr);  
11    Arrays.sort(tArr);  
12  
13    return Arrays.equals(sArr, tArr);  
14 }
```

- Time Complexity: $O(n \log n)$
- Space Complexity: $O(n)$

Approach 2: Optimal

Since the strings contain **only lowercase English letters**, we can count character frequencies and compare.

```
public boolean isAnagram(String s, String t) {  
    if (s.length() != t.length()) {  
        return false;  
    }  
  
    int[] count = new int[26];  
  
    for (int i = 0; i < s.length(); i++) {  
        count[s.charAt(i) - 'a']++;  
        count[t.charAt(i) - 'a']--;  
    }  
  
    for (int c : count) {  
        if (c != 0) {  
            return false;  
        }  
    }  
  
    return true;  
}
```

- Time: $O(n)$
- Space: $O(1)$ (constant size array)



Question 3: Two Sum

Given an array of integers `nums` and an integer `target`, return the indices `i` and `j` such that `nums[i] + nums[j] == target` and `i != j`.

You may assume that every input has exactly one pair of indices `i` and `j` that satisfy the condition.

Return the answer with the smaller index first.

Example 1:

Input:

`nums = [3,4,5,6]`, `target = 7`

Output: `[0,1]`

Explanation: `nums[0] + nums[1] == 7`, so we return `[0, 1]`.

Example 2:

Input: `nums = [4,5,6]`, `target = 10`

Output: `[0,2]`

Example 3:

Input: `nums = [5,5]`, `target = 10`

Output: `[0,1]`

Constraints:

`2 <= nums.length <= 1000`

`-10,000,000 <= nums[i] <= 10,000,000`

`-10,000,000 <= target <= 10,000,000`

Only one valid answer exists.

Approach 1: Brute force

Check **every pair** of numbers to see if they sum to the target.

```
2      public int[] twoSum(int[] nums, int target) {
3          int[] res = new int[2];
4          int n = nums.length;
5
6          for (int i = 0; i < n; i++) {
7              for (int j = i + 1; j < n; j++) {
8                  if (nums[i] + nums[j] == target) {
9                      res[0] = i;
10                     res[1] = j;
11                 }
12             }
13         }
14
15         return res;
16     }
```

- Time: $O(n^2)$
- Space: $O(1)$

Approach 2: Optimal

Store numbers seen so far in a map:

- Key $\rightarrow$ number
- Value $\rightarrow$ index

For each number, check if **target - current** already exists.



```
2   public int[] twoSum(int[] nums, int target) {  
3       int[] res = new int[2];  
4       HashMap<Integer,Integer> mp = new HashMap<>();  
5       for(int i=0; i<nums.length; i++) {  
6           if(mp.containsKey(target-nums[i])) {  
7               res[0] = i;  
8               res[1] = mp.get(target-nums[i]);  
9               break;  
10          }  
11          mp.put(nums[i],i);  
12      }  
13      return res;  
14  }
```

- Time: $O(n)$
- Space: $O(n)$



Question 4: Valid Anagrams

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

There is no string in `strs` that can be rearranged to form "bat".

The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.

The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

$1 \leq \text{strs.length} \leq 104$

$0 \leq \text{strs}[i].\text{length} \leq 100$

`strs[i]` consists of lowercase English letters.

Approach :

- Sort each string → sorted string will be the same for anagrams
- Use a HashMap where:
 - Key → sorted string
 - Value → list of strings that are anagrams

```
2    public List<List<String>> groupAnagrams(String[] strs) {
3        Map<String, List<String>> map = new HashMap<>();
4
5        for (String word : strs) {
6            char[] chars = word.toCharArray();
7            Arrays.sort(chars);
8            String sortedWord = new String(chars);
9
10           if (!map.containsKey(sortedWord)) {
11               map.put(sortedWord, new ArrayList<>());
12           }
13
14           map.get(sortedWord).add(word);
15       }
16
17       return new ArrayList<>(map.values());
18   }
```

- **Time:** $O(n * k \log k)$ (sorting each string)
- **Space:** $O(n * k)$ (storing grouped strings)

Question 5: Encode and Decode strings

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Constraints:

$0 \leq \text{strs.length} < 100$

$0 \leq \text{strs}[i].\text{length} < 200$

$\text{strs}[i]$ contains any possible characters out of 256 valid ASCII characters.

Approach :

We will use the **format**:

`<length>#<string>`

Where:

- `<length>` = length of the string
- `#` = delimiter (any character that is **not interpreted**, just a marker)
- `<string>` = the original string

This ensures that even if the string contains `#`, we know exactly how many characters to read.

```
public String encode(List<String> strs) {  
    StringBuilder sb = new StringBuilder();  
  
    for (String s : strs) {  
        sb.append(s.length()).append("#").append(s);  
    }  
  
    return sb.toString();  
}
```

```
public List<String> decode(String str) {  
    List<String> result = new ArrayList<>();  
    int i = 0;  
    while (i < str.length()) {  
        // Find next delimiter '#'  
        int j = i;  
        while (str.charAt(j) != '#') {  
            j++;  
        }  
        // Extract length  
        int length = Integer.parseInt(str.substring(i, j));  
        // Extract the actual string  
        String s = str.substring(j + 1, j + 1 + length);  
        result.add(s);  
        // Move to the next string  
        i = j + 1 + length;  
    }  
    return result;  
}
```

- **Time:** $O(n * k)$ → n = number of strings, k = average string length
- **Space:** $O(n * k)$ → for encoded string and decoded list